

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
%matplotlib inline
import numpy as np
```

```
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()
print(len(X_train))
print(len(X_test))
```

```
60000
10000
```

```
X_train = np.expand_dims(X_train, -1) / 255.0
X_test = np.expand_dims(X_test, -1) / 255.0
```

```
model = models.Sequential([

    layers.Input(shape=(28,28, 1)),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

```
model.compile(optimizer='adam',
              loss = 'sparse_categorical_crossentropy',
              metrics = ['accuracy']
              )
```

```
model.fit(X_train,y_train, epochs=10)
```

```
Epoch 1/10
1875/1875 ————— 12s 4ms/step - accuracy: 0.9559 - loss: 0.1462
Epoch 2/10
1875/1875 ————— 8s 4ms/step - accuracy: 0.9857 - loss: 0.0464
Epoch 3/10
1875/1875 ————— 8s 4ms/step - accuracy: 0.9895 - loss: 0.0331
Epoch 4/10
1875/1875 ————— 8s 4ms/step - accuracy: 0.9918 - loss: 0.0254
Epoch 5/10
1875/1875 ————— 8s 4ms/step - accuracy: 0.9941 - loss: 0.0193
Epoch 6/10
1875/1875 ————— 8s 4ms/step - accuracy: 0.9943 - loss: 0.0168
Epoch 7/10
```

```
1875/1875 ————— 8s 4ms/step - accuracy: 0.9956 - loss: 0.0136
Epoch 8/10
1875/1875 ————— 8s 4ms/step - accuracy: 0.9962 - loss: 0.0119
Epoch 9/10
1875/1875 ————— 8s 4ms/step - accuracy: 0.9969 - loss: 0.0094
Epoch 10/10
1875/1875 ————— 8s 4ms/step - accuracy: 0.9969 - loss: 0.0097
<keras.src.callbacks.history.History at 0x299bfad9690>
```

```
model.evaluate(X_test,y_test)
```

```
313/313 ————— 1s 2ms/step - accuracy: 0.9914 - loss: 0.0368
[0.036754682660102844, 0.9914000034332275]
```

Start coding or [generate](#) with AI.